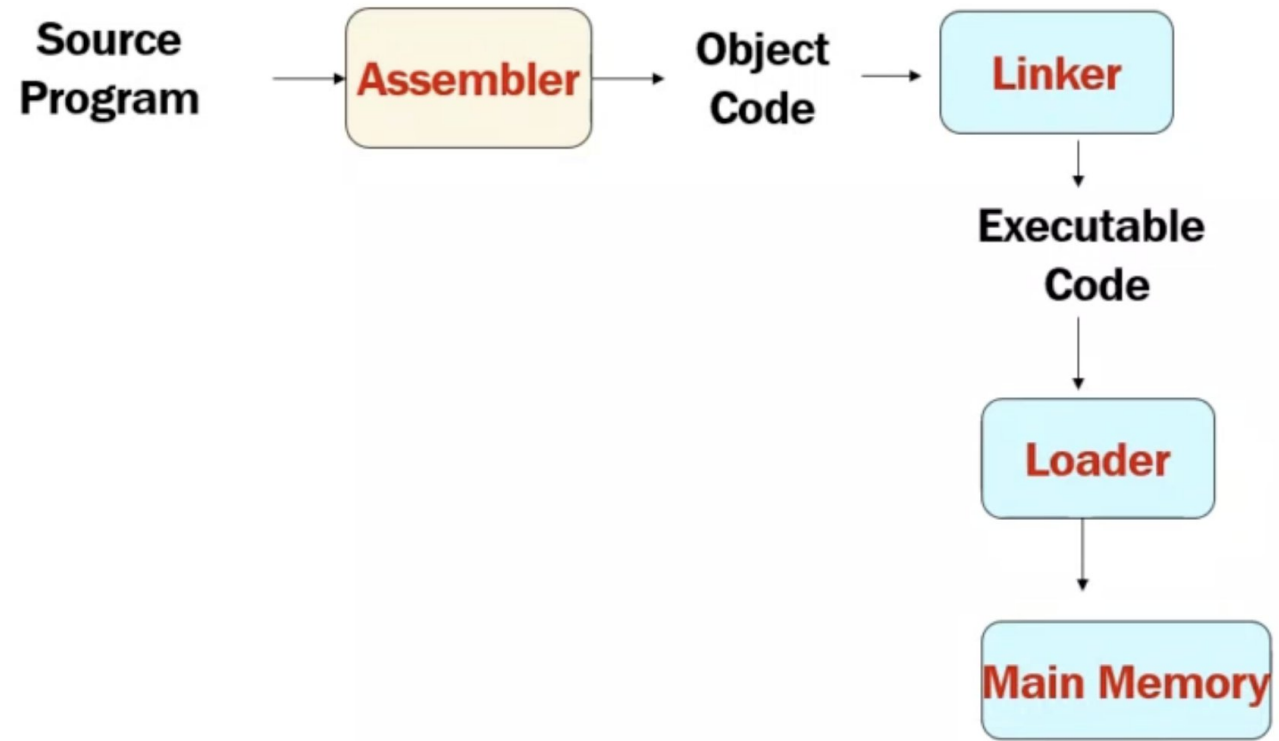
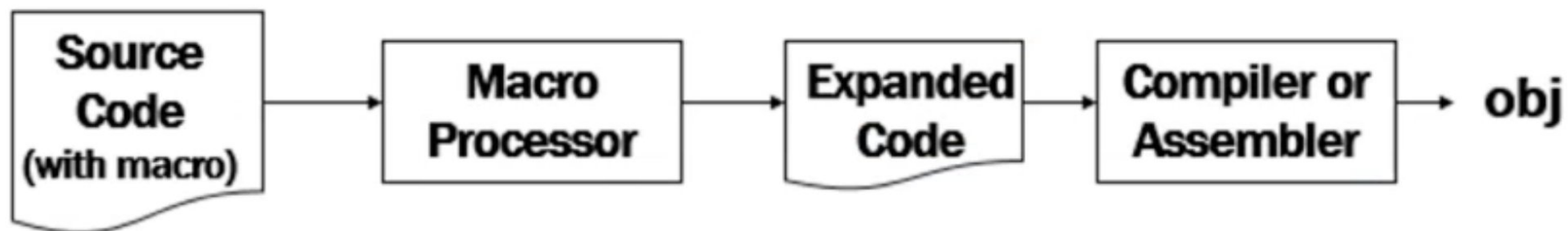


Macroprocessor





Feature	Function Call	Macro Call
Mechanism	Transfers control to a separate block of code (function/subroutine)	Replaces the macro call with the macro body (text substitution)
Execution	Happens at runtime	Happens at assembly/compile time
Code memory	One copy of function code, reused each call	Each macro call inserts a new copy of the macro body (code expansion)
Overhead	Call/return overhead (saving PC, stack management, etc.)	No call/return overhead (code is inline)
Parameter passing	Parameters passed via stack/registers, evaluated at runtime	Parameters substituted as text during macro expansion

Introduction

- A macro instruction (**macro**) is a notational **convenience** for the programmer.
- It allows the programmer to write **shorthand version** of a program(module programming)
- The macro processor **replaces** each macro instruction with the corresponding group of **source language** statements (**expanding**)
- Normally, it performs **no analysis of the text** it handles.
- It **does not concern the meaning** of the involved statements during macro expansion.
- The design of a macro processor generally is **machine independent**!

Introduction to Macro Processors

- A *macro instruction* (*macro*) is a notational convenience for the programmer.
 - Allow the programmer to write a *shorthand* version of a program
- A *macro* represents a commonly used group of statements in the source programming language.
- Expanding the macros
 - The macro processor replaces each macro instruction with the corresponding group of source language statements.

- A macro processor
 - Essentially involve the substitution of one group of characters or lines for another.
 - Normally, it performs **no analysis** of the text it handles.
 - It doesn't concern the **meaning** of the involved statements during macro expansion
- The design of a macro processor generally is **machine independent**.

- Three examples of actual macro processors:
 - A macro processor designed for use by assembler language programmers
 - Used with a high-level programming language
 - General-purpose macro processor, which is not tied to any particular language



4.1 Basic Macro Processor Functions

- Macro Definition
- Macro Expansion
- Macro Invocation

4.1 Basic Macro Processors Functions

- *Macro processor* should processes the
 - **Macro definitions**
 - Define macro name, group of instructions
 - **Macro invocation (macro calls)**
 - A body is simply copied or substituted at the point of call
 - Expansion with substitution of parameters
 - Arguments are textually substituted for the parameters
 - The resulting procedure body is textually substituted for the call

Macro Definition

- Two new assembler directives are used in macro definition

MACRO: identify the beginning of a macro definition

MEND: identify the end of a macro definition

- Each parameter begins with '&'

- Prototype for the macro

macro_name MACRO parameters

:

Body

:

MEND

- Body: the statements that will be generated as the expansion of the macro.

Macro Expansion

Source

```
STRG  MACRO
      STA  DATA1
      STB  DATA2
      STX  DATA3
      MEND
```

```
·
STRG
·
STRG
·
·
```

Expanded source

```
·
·
·

{ STA  DATA1
  STB  DATA2
  STX  DATA3

·

{ STA  DATA1
  STB  DATA2
  STX  DATA3

·
```

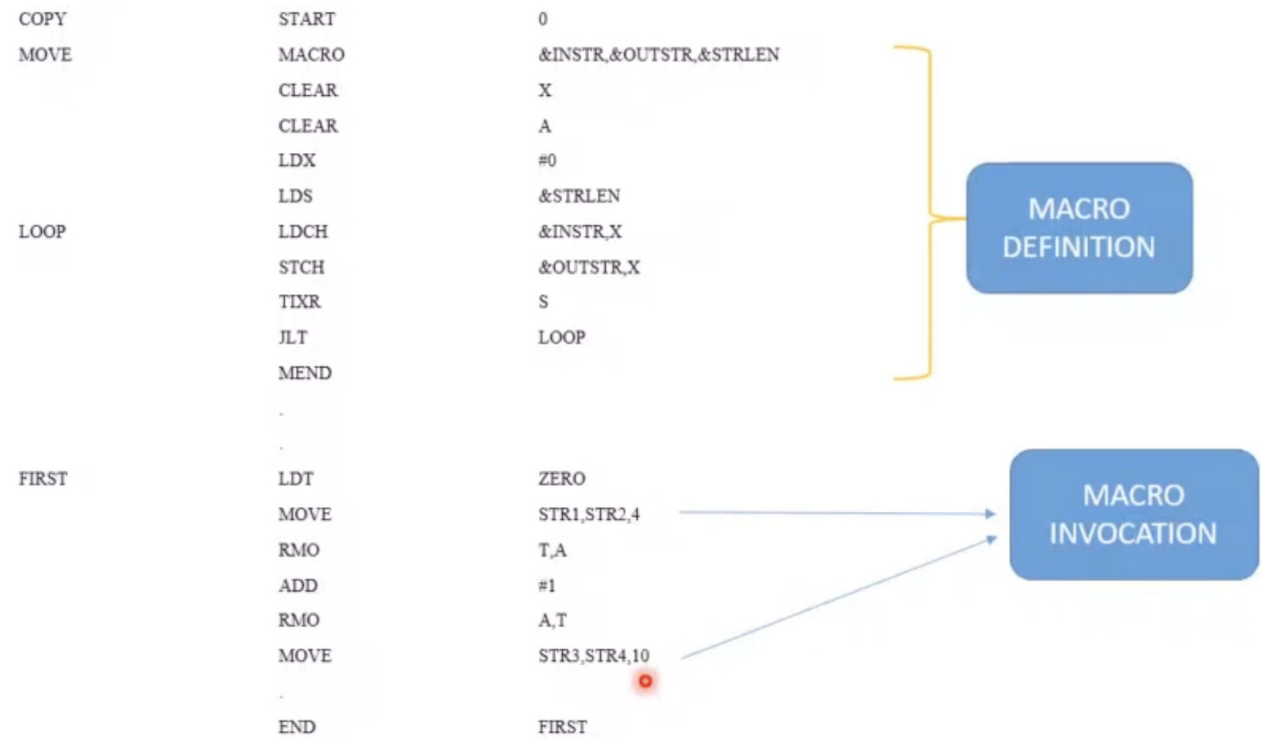
Macro Invocation

- A macro invocation statement (a macro call) gives the name of the macro instruction being invoked and the arguments to be used in expanding the macro.

`macro_name a1, a2, ...`

- Difference between macro call (invocation) and procedure call
 - Macro call: statements of the macro body are expanded each time the macro is invoked.
 - Procedure call: statements of the subroutine appear only one, regardless of how many times the subroutine is called.

Programming Example



Programming Example

COPY	START	0
FIRST	LDT	ZERO
	MOVE	STR1,STR2,4
	CLEAR	X
	CLEAR	A
	LDX	#0
	LDS	4
LOOP	LDCH	STR1,X
	STCH	STR2,X
	TIXR	S
	JLT	LOOP
	RMO	T,A
	ADD	#1
	RMO	A,T
	MOVE	STR3,STR4,10
	CLEAR	X
	CLEAR	A
	LDX	#0
	LDS	10
LOOP	LDCH	STR3,X
	STCH	STR4,X
	TIXR	S
	JLT	LOOP
	.	
	END	FIRST

COPY	START	0
MOVE	MACRO	&INSTR,&OUTSTR,&STRLEN
	CLEAR	X
	CLEAR	A
	LDX	#0
	LDS	&STRLEN
LOOP	LDCH	&INSTR,X
	STCH	&OUTSTR,X
	TIXR	S
	JLT	LOOP
	MEND	
	.	
	.	
FIRST	LDT	ZERO
	MOVE	STR1,STR2,4
	RMO	T,A
	ADD	#1
	RMO	A,T
	MOVE	STR3,STR4,10
	.	
	END	FIRST

Macro definition and Expansion

- Fig. 4.1 shows an example of a SIC/XE program using macro instructions.
 - RDBUFF and WRBUFF
 - MACRO and MEND
 - RDBUFF is name
 - *Parameters* of the macro instruction, each parameter begins with the character &.
 - Macro invocation statement and the arguments to be used in expanding the macro.
- Fig. 4.2 shows the output that would be generated.

Line	Source statement
5	COPY START 0 COPY FILE FROM INPUT TO OUTPUT
10	RDBUFF MACRO &INDEV, &BUFADR, &RECLTH
15	.
20	MACRO TO READ RECORD INTO BUFFER
25	.
30	CLEAR X CLEAR LOOP COUNTER
35	CLEAR A
40	CLEAR S
45	+LDT #4096 SET MAXIMUM RECORD LENGTH
50	TD =X'&INDEV' TEST INPUT DEVICE
55	JEQ *-3 LOOP UNTIL READY
60	RD =X'&INDEV' READ CHARACTER INTO REG A
65	COMPR A, S TEST FOR END OF RECORD
70	JEQ *+11 EXIT LOOP IF EOR
75	STCH &BUFADR, X STORE CHARACTER IN BUFFER
80	TIXR T LOOP UNLESS MAXIMUM LENGTH
85	JLT *-19 HAS BEEN REACHED
90	STX &RECLTH SAVE RECORD LENGTH
95	MEND
100	WRBUFF MACRO &OUTDEV, &BUFADR, &RECLTH
105	.
110	MACRO TO WRITE RECORD FROM BUFFER
115	.
120	CLEAR X CLEAR LOOP COUNTER
125	LDT &RECLTH
130	LDCH &BUFADR, X GET CHARACTER FROM BUFFER
135	TD =X'&OUTDEV' TEST OUTPUT DEVICE
140	JEQ *-3 LOOP UNTIL READY
145	WD =X'&OUTDEV' WRITE CHARACTER
150	TIXR T LOOP UNTIL ALL CHARACTERS
155	JLT *-14 HAVE BEEN WRITTEN
160	MEND
165	.
170	MAIN PROGRAM
175	.
180	FIRST STL RETADR SAVE RETURN ADDRESS
190	CLOOP RDBUFF F1, BUFFER, LENGTH READ RECORD INTO BUFFER
195	LDA LENGTH TEST FOR END OF FILE
200	COMP #0
205	JEQ ENDFIL EXIT IF EOF FOUND
210	WRBUFF 05, BUFFER, LENGTH WRITE OUTPUT RECORD
215	J CLOOP LOOP
220	ENDFIL WRBUFF 05, EOF, THREE INSERT EOF MARKER
225	J @RETADR
230	EOF BYTE C'EOF'
235	THREE WORD 3
240	RETADR RESW 1
245	LENGTH RESW 1 LENGTH OF RECORD
250	BUFFER RESB 4096 4096-BYTE BUFFER AREA
255	END FIRST

Figure 4.1 Use of macros in a SIC/XE program.

5	COPY	START	0	COPY FILE FROM INPUT TO OUTPUT
10	RDBUFF	MACRO	&INDEV, &BUFADR, &RECLTH	
15	.			
20	.	MACRO	TO READ RECORD INTO BUFFER	
25	.			
30		CLEAR	X	CLEAR LOOP COUNTER
35		CLEAR	A	
40		CLEAR	S	
45		+LDT	#4096	SET MAXIMUM RECORD LENGTH
50		TD	=X'&INDEV'	TEST INPUT DEVICE
55		JEQ	*-3	LOOP UNTIL READY
60		RD	=X'&INDEV'	READ CHARACTER INTO REG A
65		COMPR	A,S	TEST FOR END OF RECORD
70		JEQ	*+11	EXIT LOOP IF EOR
75		STCH	&BUFADR,X	STORE CHARACTER IN BUFFER
80		TIXR	T	LOOP UNLESS MAXIMUM LENGTH
85		JLT	*-19	HAS BEEN REACHED
90		STX	&RECLTH	SAVE RECORD LENGTH
95		MEND		

100	WRBUFF	MACRO	&OUTDEV, &BUFADR, &RECLTH	
105	.			
110	.	MACRO TO WRITE RECORD FROM BUFFER		
115	.			
120		CLEAR	X	CLEAR LOOP COUNTER
125		LDT	&RECLTH	
130		LDCH	&BUFADR, X	GET CHARACTER FROM BUFFER
135		TD	=X' &OUTDEV'	TEST OUTPUT DEVICE
140		JEQ	*-3	LOOP UNTIL READY
145		WD	=X' &OUTDEV'	WRITE CHARACTER
150		TLXR	T	LOOP UNTIL ALL CHARACTERS
155		JLT	*-14	HAVE BEEN WRITTEN
160		MEND		

165	.			
170	.	MAIN PROGRAM		
175	.			
180	FIRST	STL	RETADR	SAVE RETURN ADDRESS
190	CLOOP	RDBUFF	F1,BUFFER,LENGTH	READ RECORD INTO BUFFER
195		LDA	LENGTH	TEST FOR END OF FILE
200		COMP	#0	
205		JEQ	ENDFIL	EXIT IF EOF FOUND
210		WRBUFF	05,BUFFER,LENGTH	WRITE OUTPUT RECORD
215		J	CLOOP	LOOP
220	ENDFIL	WRBUFF	05,EOF,THREE	INSERT EOF MARKER
225		J	@RETADR	
230	EOF	BYTE	C'EOF'	
235	THREE	WORD	3	
240	RETADR	RESW	1	
245	LENGTH	RESW	1	LENGTH OF RECORD
250	BUFFER	RESB	4096	4096-BYTE BUFFER AREA
255		END	FIRST	

180	FIRST	STL	RETADR	SAVE RETURN ADDRESS
190	CLOOP	RDBUFF	F1,BUFFER,LENGTH	READ RECORD INTO BUFFER
195		LDA	LENGTH	TEST FOR END OF FILE
200		COMP	#0	
205		JEQ	ENDFIL	EXIT IF EOF FOUND
210		WRBUFF	05,BUFFER,LENGTH	WRITE OUTPUT RECORD
215		J	CLOOP	LOOP
220	ENDFIL	WRBUFF	05,EOF,THREE	INSERT EOF MARKER
225		J	@RETADR	
230	EOF	BYTE	C'EOF'	
235	THREE	WORD	3	
240	RETADR	RESW	1	
245	LENGTH	RESW	1	LENGTH OF RECORD
250	BUFFER	RESB	4096	4096-BYTE BUFFER AREA
255		END	FIRST	

Figure 4.1 Use of macros in a SIC/XE program.


```

10  RDBUFF  MACRO    &INDEV, &BUFADR, &RECLTH
15  .
20  .      MACRO TO READ RECORD INTO BUFFER
25  .
30          CLEAR   X
35          CLEAR   A
40          CLEAR   S
45          +LDT     #4096
50          TD       =X'&INDEV'
55          JEQ      *-3
60          RD       =X'&INDEV'
65          COMPR    A, S
70          JEQ      *+11
75          STCH     &BUFADR, X
80          TIXR     T
85          JLT      *-19
90          STX      &RECLTH
95          MEND

```

```

5  COPY    START    0
180 FIRST  STL      RETADR
190 .CLOOP  RDBUFF   F1, BUFFER, LENGTH
190a CLOOP  CLEAR    X
190b        CLEAR    A
190c        CLEAR    S
190d        +LDT     #4096
190e        TD       =X'F1'
190f        JEQ      *-3
190g        RD       =X'F1'
190h        COMPR    A, S
190i        JEQ      *+11
190j        STCH     BUFFER, X
190k        TIXR     T
190l        JLT      *-19
190m        STX      LENGTH
195        LDA      LENGTH
200        COMP     #0
205        JEQ      ENDFIL  17

```

```

100 WRBUFF MACRO &OUTDEV, &BUFADR, &RECLTH
105 .
110 . MACRO TO WRITE RECORD FROM BUFFER
115 .
120 CLEAR X
125 LDT &RECLTH
130 LDCH &BUFADR, X
135 TD =X'&OUTDEV'
140 JEQ *-3
145 WD =X'&OUTDEV'
150 TIXR T
155 JLT *-14
160 MEND
165 .
170 . MAIN PROGRAM
175 .

```

```

210
210a
210b
210c
210d
210e
210f
210g
210h
215

```

```

WRBUFF 05, BUFFER, LENGTH
CLEAR X
LDT LENGTH
LDCH BUFFER, X
TD =X'05'
JEQ *-3
WD =X'05'
TIXR T
JLT *-14
J CLOOP

```

```

100 WRBUFF MACRO &OUTDEV,&BUFADR,&RECLTH
105 .
110 . MACRO TO WRITE RECORD FROM BUFFER
115 .
120 CLEAR X
125 LDT &RECLTH
130 LDCH &BUFADR,X
135 TD =X'&OUTDEV'
140 JEQ *-3
145 WD =X'&OUTDEV'
150 TIXR T
155 JLT *-14
160 MEND
165 .
170 . MAIN PROGRAM
175 .

```

```

220 .ENDFIL WRBUFF 05,EOF,THREE
220a ENDFIL CLEAR X
220b LDT THREE
220c LDCH EOF,X
220d TD =X'05'
220e JEQ *-3
220f WD =X'05'
220g TIXR T
220h JLT *-14
225 J @RETADR
230 EOF BYTE C'EOF'
235 THREE WORD 3
240 RETADR RESW 1
245 LENGTH RESW 1
250 BUFFER RESB 4096
255 END FIRST

```

```

5      COPY      START      0
180    FIRST     STL        RETADR
190    .CLOOP    RDBUFF     F1,BUFFER,LENGTH
190a   CLOOP     CLEAR      X
190b           CLEAR      A
190c           CLEAR      S
190d           +LDT        #4096
190e           TD          =X'F1'
190f           JEQ         *-3
190g           RD          =X'F1'
190h           COMPR       A,S
190i           JEQ         *+11
190j           STCH        BUFFER,X
190k           TIXR        T
190l           JLT         *-19
190m           STX         LENGTH
195           LDA          LENGTH
200           COMP        #0
205           JEQ         ENDFIL

```

```

210    WRBUFF    05,BUFFER,LENGTH
210a   CLEAR     X
210b   LDT       LENGTH
210c   LDCH      BUFFER,X
210d   TD        =X'05'
210e   JEQ       *-3
210f   WD        =X'05'
210g   TIXR      T
210h   JLT       *-14
215    J         CLOOP
220    .ENDFIL   WRBUFF    05,EOF,THREE
220a   ENDFIL    CLEAR     X
220b   LDT       THREE
220c   LDCH      EOF,X
220d   TD        =X'05'
220e   JEQ       *-3
220f   WD        =X'05'
220g   TIXR      T
220h   JLT       *-14
225    J         @RETADR
230    EOF       BYTE      C'EOF'
235    THREE     WORD      3
240    RETADR    RESW      1
245    LENGTH    RESW      1
250    BUFFER    RESB      4096
255    END       FIRST

```

Changes observed

- The macro instruction definitions have been deleted, they are not useful once the expanded.
- Each macro invocation statement has been expanded into statements that form the body of macro.
- The arguments from macro invocation are substituted as parameters in macro prototype, with exact position.
- The parameters have been updated in macro prototype.

- Generic Labelling

```
JEQ *-3      for    JEQ WLOOP
JLT *-14     for    JLT  WLOOP
```



Macro Expansion

- Each macro invocation statement will be **expanded** into the statements that form the **body** of the macro.

- Arguments from the macro invocation are **substituted** for the parameters in the macro prototype.
 - The arguments and parameters are associated with one another according to their **positions**.
 - The first argument in the macro invocation corresponds to the first parameter in the macro prototype, etc.

Two-pass macro processor

- Pass1: Recognize macro definitions
- Pass2: Recognize macro calls
- **Two-pass macro processor would not allow the body of one macro instruction to contain definitions of other macros** - Nested macro definitions are not allowed

recursive macro definition

1	MACROS	MACRO	{Defines SIC standard version macros}
2	RDBUFF	MACRO	&INDEV,&BUFADR,&RECLTH
		.	
		.	{SIC standard version}
		.	
3		MEND	{End of RDBUFF}
4	WRBUFF	MACRO	&OUTDEV,&BUFADR,&RECLTH
		.	
		.	{SIC standard version}
5		MEND	{End of WRBUFF}
		.	
		.	
6		MEND	{End of MACROS}

1	MACROX	MACRO	{Defines SIC/XE macros}
2	RDBUFF	MACRO	&INDEV,&BUFADR,&RECLTH
		.	
		.	{SIC/XE version}
		.	
3		MEND	{End of RDBUFF}
4	WRBUFF	MACRO	&OUTDEV,&BUFADR,&RECLTH
		.	
		.	{SIC/XE version}
		.	
5		MEND	{End of WRBUFF}
		.	
		.	
6		MEND	{End of MACROX}

Comparison

Two-pass algorithm

- Pass1: Recognize macro definitions
- Pass2: Recognize macro calls ○
- Nested macro definitions are not allowed

One-pass algorithm

- Every macro must be defined before it is called
- One-pass processor can alternate between macro definition and macro expansion
- Nested macro definitions are allowed but nested calls are not

One-pass macro processor

- A one-pass macro processor that **alternate** between **macro definition** and **macro expansion** in a recursive way is able to handle recursive macro definition.
- Restriction
 - The definition of a macro must appear in the source program before any statements that invoke that macro.
 - This restriction does not create any real inconvenience.

Data structures for one-pass macro processor

- DEFTAB (definition table)

- Stores the macro definition including macro prototype and macro body
- Comment lines are omitted.
- References to the macro instruction parameters are converted to a positional notation for efficiency in substituting arguments.

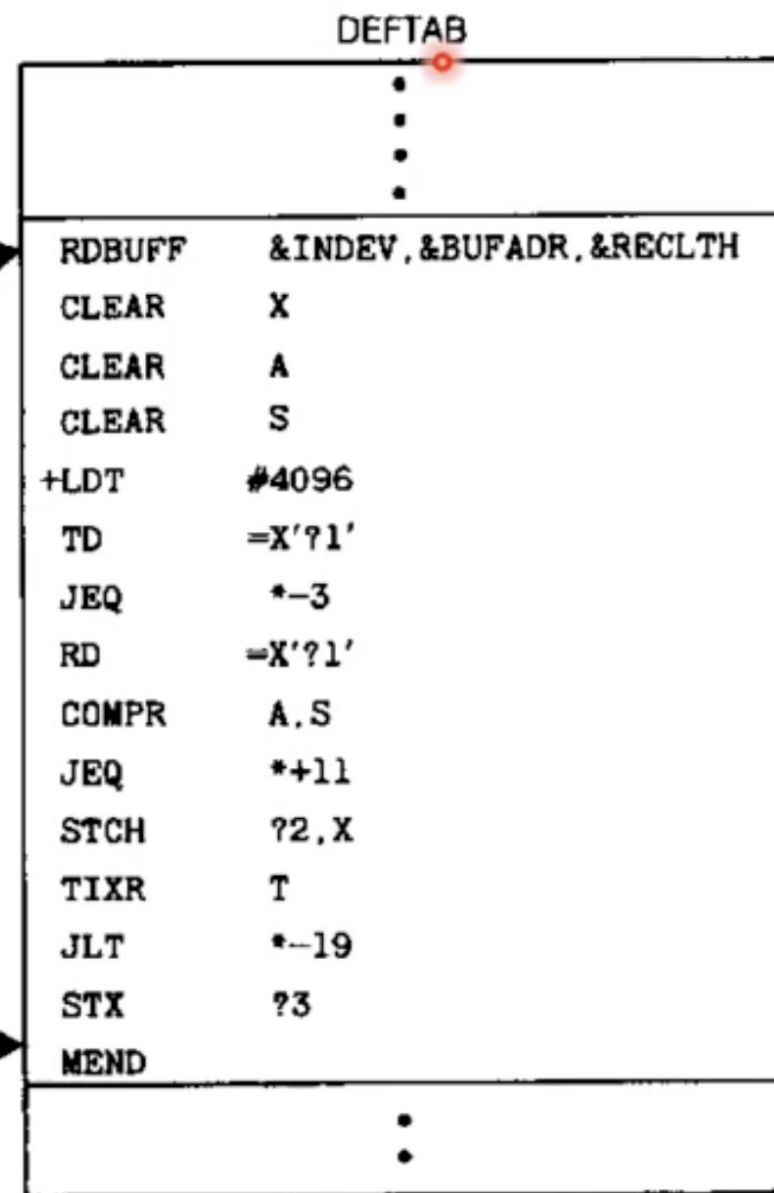
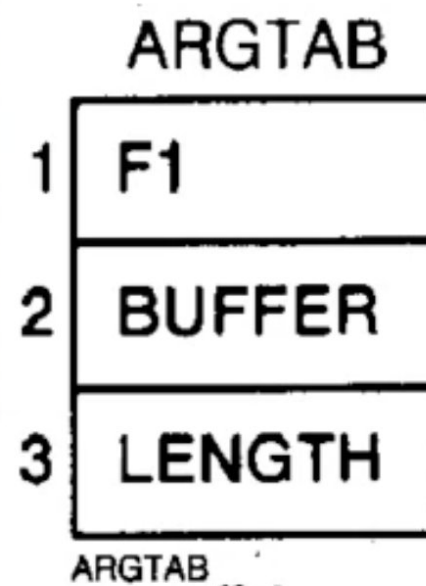
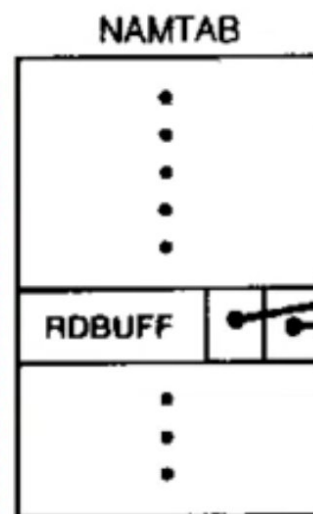
- NAMTAB

- Stores macro names
- Serves as an index to DEFTAB
 - Pointers to the beginning and the end of the macro definition (DEFTAB)

- ARGTAB

- Stores the arguments of macro invocation according to their positions in the argument list
- As the macro is expanded, arguments from ARGTAB are substituted for the corresponding parameters in the macro body.

5	COPY	START	0
180	FIRST	STL	RETADR
190	.CLOOP	RDBUFF	F1, BUFFER, LENGTH
190a	CLOOP	CLEAR	X
190b		CLEAR	A
190c		CLEAR	S
190d		+LDT	#4096
190e		TD	=X'F1'
190f		JEQ	*-3
190g		RD	=X'F1'
190h		COMPR	A, S
190i		JEQ	*+11
190j		STCH	BUFFER, X
190k		TIXR	T
190l		JLT	*-19
190m		STX	LENGTH
195		LDA	LENGTH
200		COMP	#0
205		JEQ	ENDFIL



(b)

(a)

Machine Independent Macroprocessor Features

MACHINE INDEPENDENT MACRO PROCESSOR FEATURES

- Concatenation of Macro Parameters
- Generation of Unique Labels
- Conditional Macro Expansion
- Keyword Macro Parameters

Concatenation of Macro Parameters

- Most Macro processors allow parameters to be concatenated with other character strings
- Suppose for example a program contains on series of variables named by the symbols XA1, XA2, XA3,.....and another series named by XB1, XB2, XB3,...etc.
- If similar processing is to be performed on each series of variables the programmer might want to incorporate this processing into a macro instruction.

▪ ***& : indicates beginning of macro parameter.***

→ : indicates end of macro parameter.

Eg:- LDA X&ID→1

SUM MACRO

&ID

LDA

X&ID→1

ADD

X&ID→2

ADD

X&ID→3

STA

X&ID→5



SUM A



LDA XA1

ADD XA2

ADD XA3

STA XA5

SUM BETA



LDA XBETA1

ADD XBETA2

ADD XBETA3

STA XBETA5



Generation of Unique Labels

- In general macro instructions are written in such a way that the body of macros contains no labels.
- If a label were placed in a macro instruction, and if macro is invoked more than once then duplicate labels appear in the expanded version.
- To avoid this labels used within the macro body begin with the special character \$.
- The symbol beginning with \$ will be replaced by \$xx where xx is a two character alphanumeric counter of the number of macro instructions expanded.
- For first macro expansion in a program xx will have the value AA and for succeeding macro expansions xx will be set to AB, AC, etc.

M1	MACRO	&D1	&D2	L1	LDA	DATA1	
L1	LDA	&D1		L2	LDB	DATA2	
L2	LDB	&D2		L1	LDA	DATA3	
	MEND			L2	LDB	DATA4	
M1	DATA1, DATA2			Expanded			
M1	DATA3, DATA4			Source Program			
	Source Program						

M1	MACRO	&D1	&D2	\$AAL1	LDA	DATA1
\$L1	LDA	&D1		\$AAL2	LDB	DATA2
\$L2	LDB	&D2		\$ABL1	LDA	DATA3
	MEND			\$ABL2	LDB	DATA4
M1	DATA1, DATA2			Expanded		
M1	DATA3, DATA4			Source Program		
	Source Program					

Conditional macro Expansion

- Macro processors can also modify the sequence of statements generated for a macro expansion depending on the arguments supplied in the macro invocation.

Example

```
M1      MACRO  &COND
          .....
          IF  (&COND NE  ` `)
              Part I
          ELSE
              Part II
          ENDIF
          .....
        MEND
```

- Part I is expanded if condition part is true, otherwise part II is expanded.
- Compare operator : NE, EQ, LE, GT

Conditional Macro expansion

- Macro-time variables
 - Begins with '&' but is not a macro instruction parameter.
 - Can be used to store working values during the macro expansion.
 - Controls the macro time conditional structures.
 - Auto initialized to a value 0.
 - Values are set by macro processor directive, SET

eg:- &EORCK SET 1
 &EORCK SET &EORCK+1

25	RDBUFF	MACRO	&INDEV, &BUFADR, &RECLTH, &EOR, &MAXLTH	
26		IF	(&EOR NE '')	
27	&EORCK	SET	1	
28		ENDIF		
30		CLEAR	X	CLEAR LOOP COUNTER
35		CLEAR	A	
38		IF	(&EORCK EQ 1)	
40		LDCH	=X'&EOR'	SET EOR CHARACTER
42		RMO	A, S	
43		ENDIF		
44		IF	(&MAXLTH EQ '')	
45		+LDT	#4096	SET MAX LENGTH = 4096
46		ELSE		
47		+LDT	#&MAXLTH	SET MAXIMUM RECORD LENGTH
48		ENDIF		
50	\$LOOP	TD	=X'&INDEV'	TEST INPUT DEVICE
55		JEQ	\$LOOP	LOOP UNTIL READY
60		RD	=X'&INDEV'	READ CHARACTER INTO REG A
63		IF	(&EORCK EQ 1)	
65		COMPR	A, S	TEST FOR END OF RECORD
70		JEQ	\$EXIT	EXIT LOOP IF EOR
73		ENDIF		
75		STCH	&BUFADR, X	STORE CHARACTER IN BUFFER
80		TLXR	T	LOOP UNLESS MAXIMUM LENGTH
85		JLT	\$LOOP	HAS BEEN REACHED
90	\$EXIT	STX	&RECLTH	SAVE RECORD LENGTH
95		MEND		

Conditional Macro expansion

- Macro-time conditional statements
 - » Example: Figure 4.8
 - » *IF-ELSE-ENDIF*
- Macro-time variables
 - » any symbol that begins with the character & and that is not a macro parameter
 - » macro-time variables are initialized to 0
 - » macro-time variables can be changed with their values using SET
 - *&EORCK* SET 1

Macro Processor Design Options: Recursive Macro Expansion, General Purpose macro processors

Recursive Macro Expansion

